

OS Scheduler

Priority

SRTF

Comparison

Algorithm A

Priority Scheduling

Preemptive · Lower number = Higher priority · Tie-breaking by Arrival Time then PID

PROCESS INPUT

Scenario A

Scenario B

Scenario C

Scenario D

Clear

PID	ARRIVAL	BURST	PRIORITY
P1	0	6	2
P2	1	4	1
P3	2	2	3
P4	3	5	2

+ Add Process

▶ Run Priority

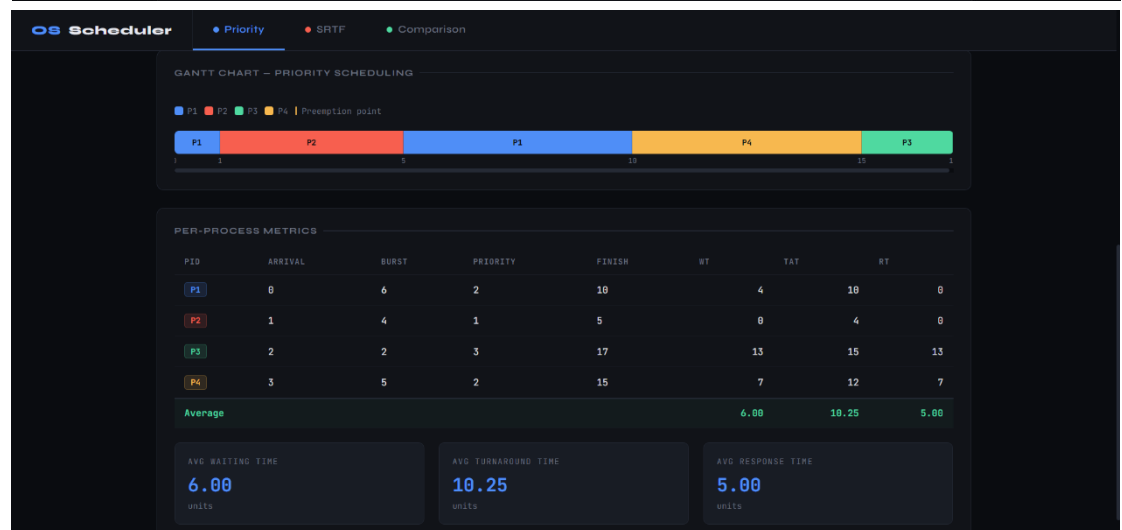
Running Priority scheduler with 4 processes...

ALGORITHM RULES

Tie-Breaking Rule:
When two processes have the *same priority*, the one with the *earlier arrival time* runs first. If arrival is also equal, lower PID wins.

Preemption: A running process is preempted when a higher-priority process arrives.

GANTT CHART – PRIORITY SCHEDULING



OS Scheduler

Priority

SRTF

Comparison

Algorithm B

SRTF Scheduling

Preemptive · Shortest Remaining Time First · Immediate preemption on new arrival

PROCESS INPUT

Scenario A

Scenario B

Scenario C

Scenario D

Clear

PID	ARRIVAL	BURST
P1	0	6
P2	1	4
P3	2	2
P4	3	5

+ Add Process

▶ Run SRTF

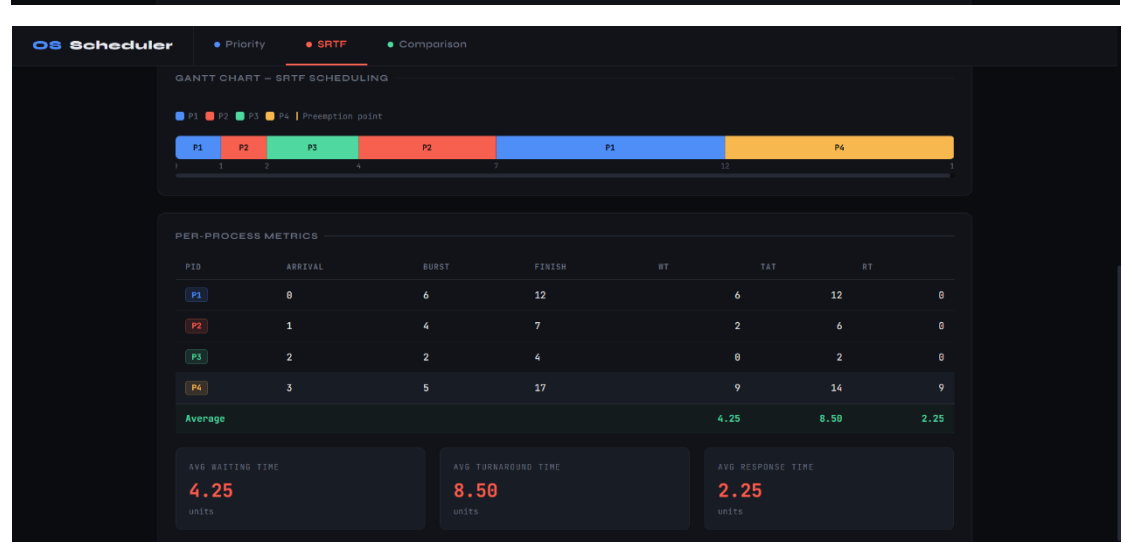
Running SRTF scheduler with 4 processes...

ALGORITHM RULES

Tie-Breaking Rule:
When two processes have the *same remaining time*, the one with the *earlier arrival time* continues (or wins). If equal, lower PID wins.

Preemption: When a new process arrives with a shorter remaining time than the current running process, it immediately preempts it. Preemption points are marked on the Gantt chart.

GANTT CHART – SRTF SCHEDULING



OS Scheduler

• Priority

• SRTF

• Comparison

C1

Priority vs SRTF – Comparison

Same workload • Both algorithms • Performance • Fairness • Trade-offs

AVG WT WINNER

SRTF

4.25 vs 6.90

AVG TAT WINNER

SRTF

8.50 vs 10.25

AVG RT WINNER

SRTF

2.25 vs 5.00

PER-PROCESS COMPARISON – WT / TAT / RT

PID	PRIORITY			SRTF		
	WT	TAT	RT	WT	TAT	RT
P1	4	6	10	12	8	8
P2	8	2	4	6	8	8
P3	13	8	15	2	13	8
P4	7	9	12	14	7	9

AVERAGE METRICS COMPARISON

OS Scheduler

• Priority

• SRTF

• Comparison

AVERAGE METRICS COMPARISON

METRIC	PRIORITY AVG	SRTF AVG	WINNER
Waiting Time (WT)	6.90	4.25	SRTF
Turnaround Time (TAT)	10.25	8.50	SRTF
Response Time (RT)	5.00	2.25	SRTF

REQUIRED ANALYSIS QUESTIONS

Q1: Which algorithm produced the lower average waiting time?

SRTF

Q2: Which algorithm produced the lower average response time?

SRTF

Q3: Did priority values improve treatment of urgent processes?

Partially – priority rules ensured ordering, but may delay short low-priority jobs unnecessarily.

Q4: Did SRTF favor short jobs more aggressively?

Yes – SRTF preempts the running process the moment a shorter job arrives, giving short processes minimal waiting time regardless of their priority.

Q5: Which algorithm would you recommend for this workload?

SRTF – wins on 3/3 metrics for this workload. Best for maximizing throughput with short jobs.

TRADE-OFF SUMMARY

PRIORITY SCHEDULING

Strengths: Policy-driven service, urgent processes guaranteed CPU. Respects external importance.
Weakness: Short low-priority processes may starve if high-priority processes keep arriving.

SRTF SCHEDULING

Strengths: Minimizes average WT and TAT globally. Optimal for throughput and short job response.
Weakness: Long processes can starve indefinitely. No notion of urgency or policy – purely burst-time driven.

OS Scheduler

• Priority

• SRTF

• Comparison

Q5: Which algorithm would you recommend for this workload?

SRTF – wins on 3/3 metrics for this workload. Best for maximizing throughput with short jobs.

REQUIRED CONCLUSION

Overall Performance: SRTF performed better on this dataset based on combined WT, TAT, and RT averages.

Metric Leaders: Priority – better at: None

SRTF – better at: WT, TAT, RT

Priority vs Burst-Time Conflict (Detected in Workload):

P2 (Priority 1, Burst 4) has higher priority but longer burst than P3 (Priority 3, Burst 2). Priority scheduling runs P2 first by policy, while SRTF would favor P3 for its shorter burst – a direct conflict between policy-based and burst-time-based service.

Policy-Based vs Burst-Time-Based Service:

Priority Scheduling is policy-driven – It enforces external importance assignments, making it suitable for real-time systems where urgency must be respected regardless of job length.
SRTF is burst-time-driven – It always runs the shortest remaining job to minimize waiting time, maximizing throughput but ignoring process importance entirely.

Starvation Risk:

Priority: Risk detected – P3 waited 13 units. Low-priority processes can starve if high-priority ones keep arriving.
SRTF: Risk detected – P4 waited 9 units. Long processes can starve when shorter jobs keep arriving.

Fairness: SRTF showed fairer overall distribution – max waiting time of 9 vs Priority's 13. SRTF's burst-time optimization tends to reduce extreme waiting cases.
Fairness in Priority depends on how priorities are assigned – biased assignments cause unfair treatment. SRTF is fair to short jobs but structurally unfair to long ones.

OS Scheduler

Priority

SRTF

Comparison

Algorithm A

Priority Scheduling

Preemptive · Lower number = Higher priority · Tie-breaking by Arrival Time then PID

PROCESS INPUT

Scenario A

Scenario B

Scenario C

Scenario D

Clear

PID	ARRIVAL	BURST	PRIORITY
P1	0	10	1
P2	1	2	4
P3	2	3	3
P4	3	1	2

+ Add Process

▶ Run Priority

Running Priority scheduler with 4 processes...

ALGORITHM RULES

Tie-Breaking Rule:
When two processes have the same priority, the one with the earlier arrival time runs first. If arrival is also equal, lower PID wins.

Preemption: A running process is preempted when a higher-priority process arrives.

GANTT CHART – PRIORITY SCHEDULING

P1

P4

P3

P2

OS Scheduler

Priority

SRTF

Comparison

GANTT CHART – PRIORITY SCHEDULING

P1

P2

P3

P4

Preemption point

PER-PROCESS METRICS

PID	ARRIVAL	BURST	PRIORITY	FINISH	WT	TAT	RT
P1	0	10	1	10	0	10	0
P2	1	2	4	16	13	15	13
P3	2	3	3	14	9	12	9
P4	3	1	2	11	7	8	7
Average					7.25	11.25	7.25

AVG WAITING TIME

7.25

units

AVG TURNAROUND TIME

11.25

units

AVG RESPONSE TIME

7.25

units

OS Scheduler

Priority

SRTF

Comparison

Algorithm B

SRTF Scheduling

Preemptive · Shortest Remaining Time First · Immediate preemption on new arrival

PROCESS INPUT

Scenario A

Scenario B

Scenario C

Scenario D

Clear

PID	ARRIVAL	BURST
P1	0	10
P2	1	2
P3	2	3
P4	3	1

+ Add Process

▶ Run SRTF

Running SRTF scheduler with 4 processes...

ALGORITHM RULES

Tie-Breaking Rule:
When two processes have the same remaining time, the one with the earlier arrival time continues (or wins). If equal, lower PID wins.

Preemption: When a new process arrives with a shorter remaining time than the current running process, it immediately preempts it. Preemption points are marked on the Gantt chart.

GANTT CHART – SRTF SCHEDULING

P1

P2

P4

P3

P1

OS Scheduler

Priority

SRTF

Comparison

GANTT CHART – SRTF SCHEDULING

P1

P2

P4

P3

Preemption point

PER-PROCESS METRICS

PID	ARRIVAL	BURST	FINISH	WT	TAT	RT
P1	0	10	16	6	16	0
P2	1	2	3	0	2	0
P3	2	3	7	2	5	2
P4	3	1	4	0	1	0
Average				2.00	6.00	0.50

AVG WAITING TIME

2.00

units

AVG TURNAROUND TIME

6.00

units

AVG RESPONSE TIME

0.50

units

OS Scheduler

Priority

SRTF

Comparison

C1

Priority vs SRTF – Comparison

Same workload · Both algorithms · Performance · Fairness · Trade-offs

AVG WT WINNER

SRTF

2.00 vs 7.25

AVG TAT WINNER

SRTF

6.00 vs 11.25

AVG RT WINNER

SRTF

0.50 vs 7.25

PER-PROCESS COMPARISON – WT / TAT / RT

PID	PRIORITY		SRTF	
	WT	TAT	WT	RT
P1	0	6	10	0
P2	13	0	2	0
P3	9	2	5	2
P4	7	0	1	0

AVERAGE METRICS COMPARISON

OS Scheduler

Priority

SRTF

Comparison

AVERAGE METRICS COMPARISON

METRIC	PRIORITY AVG	SRTF AVG	WINNER
Waiting Time (WT)	7.25	2.00	SRTF
Turnaround Time (TAT)	11.25	6.00	SRTF
Response Time (RT)	7.25	0.50	SRTF

REQUIRED ANALYSIS QUESTIONS

Q1: Which algorithm produced the lower average waiting time?

SRTF

Q2: Which algorithm produced the lower average response time?

SRTF

Q3: Did priority values improve treatment of urgent processes?

Yes – high-priority processes ran first even when not shortest, serving urgency effectively.

Q4: Did SRTF favor short jobs more aggressively?

Yes – SRTF preempts the running process the moment a shorter job arrives, giving short processes minimal waiting time regardless of their priority.

Q5: Which algorithm would you recommend for this workload?

SRTF – wins on 3/3 metrics for this workload. Best for maximizing throughput with short jobs.

TRADE-OFF SUMMARY

PRIORITY SCHEDULING

Strengths: Policy-driven service, urgent processes guaranteed CPU. Respects external importance.

Weakness: Short low-priority processes may starve if high-priority processes keep arriving.

SRTF SCHEDULING

Strengths: Minimizes average RT and TAT globally. Optimal for throughput and short-job response.

Weakness: Long processes can starve indefinitely. No notion of urgency or policy – purely burst-time driven.

OS Scheduler

Priority

SRTF

Comparison

Q5: Which algorithm would you recommend for this workload?

SRTF – wins on 3/3 metrics for this workload. Best for maximizing throughput with short jobs.

REQUIRED CONCLUSION

Overall Performance: SRTF performed better on this dataset based on combined WT, TAT, and RT averages.

Metric Leaders: Priority – better at: None

SRTF – better at: WT, TAT, RT

Priority vs Burst-Time Conflict (Detected in Workload):

P1 (Priority 1, Burst 10) has higher priority but longer burst than P2 (Priority 4, Burst 2). Priority scheduling runs P1 first by policy, while SRTF would favor P2 for its shorter burst – a direct conflict between policy-based and burst-time-based service.

Policy-Based vs Burst-Time-Based Service:

Priority Scheduling is policy-driven – it enforces external importance assignments, making it suitable for real-time systems where urgency must be respected regardless of job length.

SRTF is burst-time-driven – it always runs the shortest remaining job to minimize waiting time, maximizing throughput but ignoring process importance entirely.

Starvation Risk:

Priority: Risk detected – P2 waited 13 units. Low-priority processes can starve if high-priority ones keep arriving.

SRTF: Risk detected – P1 waited 6 units. Long processes can starve when shorter jobs keep arriving.

Fairness: SRTF showed fairer overall distribution – max waiting time of 6 vs Priority's 13. SRTF's burst-time optimization tends to reduce extreme waiting cases.

Fairness in Priority depends on how priorities are assigned – biased assignments cause unfair treatment. SRTF is fair to short jobs but structurally unfair to long ones.

OS Scheduler

Priority

SRTF

Comparison

Algorithm A

Priority Scheduling

Preemptive · Lower number = Higher priority · Tie-breaking by Arrival Time then PID

PROCESS INPUT

Scenario A

Scenario B

Scenario C

Scenario D

Clear

PID	ARRIVAL	BURST	PRIORITY
P1	0	15	5
P2	1	3	1
P3	2	2	1
P4	3	4	1

+ Add Process

▶ Run Priority

Running Priority scheduler with 4 processes...

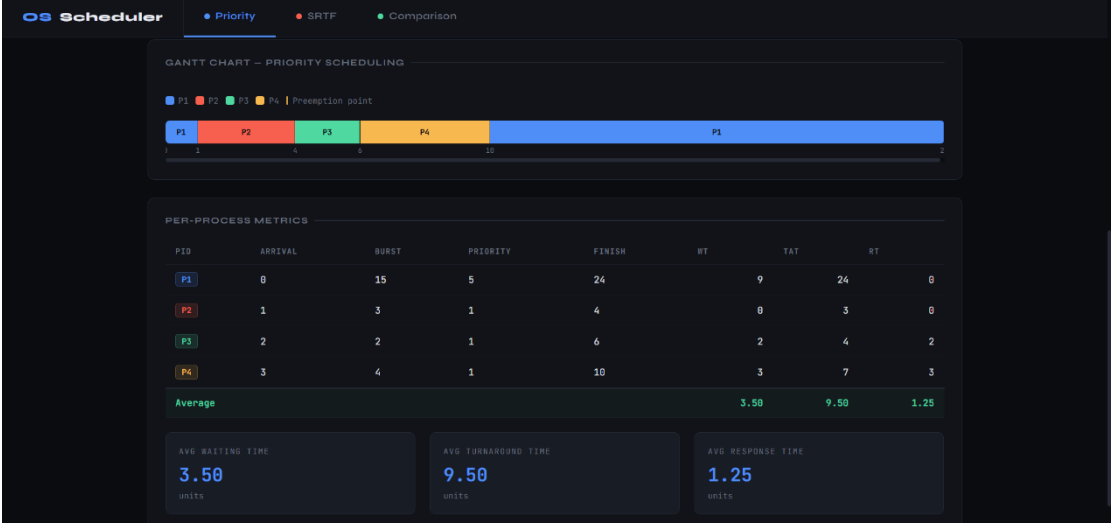
ALGORITHM RULES

Tie-Breaking Rule:

When two processes have the *same priority*, the one with the *earlier arrival time* runs first. If arrival is also equal, lower *PID* wins.

Preemption: A running process is preempted when a higher-priority process arrives.

GANTT CHART – PRIORITY SCHEDULING



OS Scheduler

Priority

SRTF

Comparison

Algorithm B

SRTF Scheduling

Preemptive · Shortest Remaining Time First · Immediate preemption on new arrival

PROCESS INPUT

Scenario A

Scenario B

Scenario C

Scenario D

Clear

PID	ARRIVAL	BURST
P1	0	15
P2	1	3
P3	2	2
P4	3	4

+ Add Process

▶ Run SRTF

Running SRTF scheduler with 4 processes...

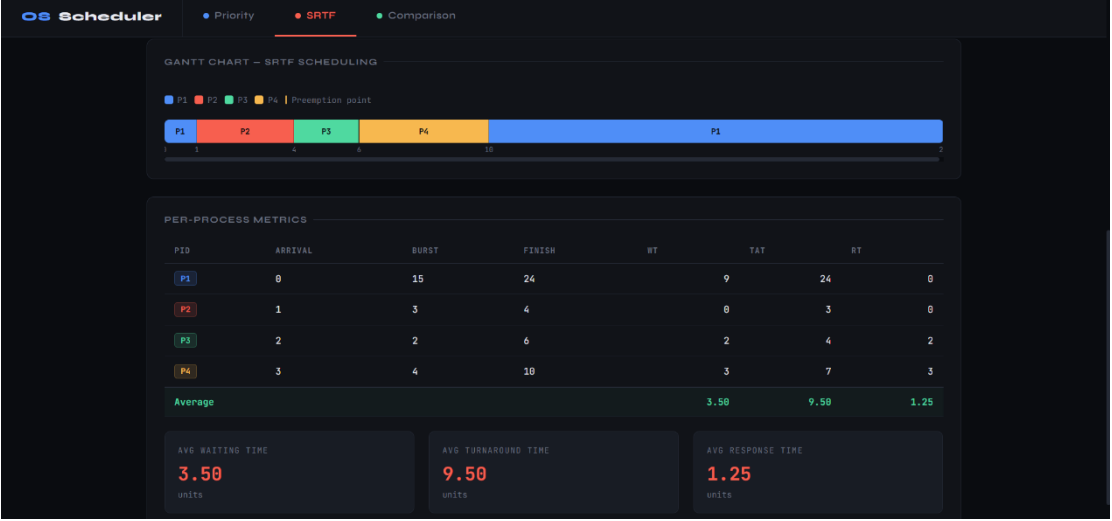
ALGORITHM RULES

Tie-Breaking Rule:

When two processes have the *same remaining time*, the one with the *earlier arrival time* continues (or wins). If equal, lower *PID* wins.

Preemption: When a new process arrives with a shorter remaining time than the current running process, it immediately preempts it. Preemption points are marked on the Gantt chart.

GANTT CHART – SRTF SCHEDULING



OS Scheduler

Priority

SRTF

Comparison

C1

Priority vs SRTF – Comparison

Same workload · Both algorithms · Performance · Fairness · Trade-offs

AVG WT WINNER

Priority

3.50 vs 3.50

AVG TAT WINNER

Priority

9.50 vs 9.50

AVG RT WINNER

Priority

1.25 vs 1.25

PER-PROCESS COMPARISON – WT / TAT / RT

PID	PRIORITY			SRTF		
	WT	TAT	RT	WT	TAT	RT
P1	9	9	24	24	8	8
P2	8	8	3	3	8	8
P3	2	2	4	4	2	2
P4	3	3	7	7	3	3

AVERAGE METRICS COMPARISON

Metric	Priority	SRTF	Winner
Waiting Time (WT)	3.50	3.50	Priority
Turnaround Time (TAT)	9.50	9.50	Priority
Response Time (RT)	1.25	1.25	Priority

OS Scheduler

Priority

SRTF

Comparison

AVERAGE METRICS COMPARISON

Metric	Priority Avg	SRTF Avg	Winner
Waiting Time (WT)	3.50	3.50	Priority
Turnaround Time (TAT)	9.50	9.50	Priority
Response Time (RT)	1.25	1.25	Priority

REQUIRED ANALYSIS QUESTIONS

Q1: Which algorithm produced the lower average waiting time?

Priority

Q2: Which algorithm produced the lower average response time?

Priority

Q3: Did priority values improve treatment of urgent processes?

Partially – priority rules ensured ordering, but may delay short low-priority jobs unnecessarily.

Q4: Did SRTF favor short jobs more aggressively?

Yes – SRTF preempts the running process the moment a shorter job arrives, giving short processes minimal waiting time regardless of their priority.

Q5: Which algorithm would you recommend for this workload?

Priority Scheduling – wins on 3/3 metrics for this workload. Best when process urgency matters.

TRADE-OFF SUMMARY

PRIORITY SCHEDULING

Strengths: Policy-driven service, urgent processes guaranteed CPU. Respects external importance.

Weakness: Short low-priority processes may starve if high-priority processes keep arriving.

SRTF SCHEDULING

Strengths: Minimizes average WT and TAT globally. Optimal for throughput and short-job response.

Weakness: Long processes can starve indefinitely. No notion of urgency or policy – purely burst-time driven.

OS Scheduler

Priority

SRTF

Comparison

Q1: Which algorithm produced the lower average waiting time?

Yes – SRTF preempts the running process the moment a shorter job arrives, giving short processes minimal waiting time regardless of their priority.

Q5: Which algorithm would you recommend for this workload?

Priority Scheduling – wins on 3/3 metrics for this workload. Best when process urgency matters.

Q4: Did SRTF favor short jobs more aggressively?

Yes – SRTF preempts the running process the moment a shorter job arrives, giving short processes minimal waiting time regardless of their priority.

Q3: Did priority values improve treatment of urgent processes?

Partially – priority rules ensured ordering, but may delay short low-priority jobs unnecessarily.

Q2: Which algorithm produced the lower average response time?

Priority

Q1: Which algorithm produced the lower average waiting time?

Priority

of urgency or policy – purely burst-time driven.

REQUIRED CONCLUSION

Overall Performance: Priority Scheduling performed better on this dataset based on combined WT, TAT, and RT averages.

Metric Leaders: Priority – better at: WT, TAT, RT
SRTF – better at: None

Policy-Based vs Burst-Time-Based Service:
Priority Scheduling is policy-driven – It enforces external importance assignments, making it suitable for real-time systems where urgency must be respected regardless of job length.
SRTF is burst-time-driven – It always runs the shortest remaining job to minimize waiting time, maximizing throughput but ignoring process importance entirely.

Starvation Risk:
Priority: Risk detected – P1 waited 9 units. Low-priority processes can starve if high-priority ones keep arriving.
SRTF: Risk detected – P1 waited 9 units. Long processes can starve when shorter jobs keep arriving.

Fairness: Priority Scheduling showed fairer distribution – max waiting time of 9 vs SRTF's 9. However, fairness in Priority depends heavily on the priority assignment.
Fairness in Priority depends on how priorities are assigned – biased assignments cause unfair treatment. SRTF is fair to short jobs but structurally unfair to long ones.

OS Scheduler

Priority

SRTF

Comparison

Algorithm A

Priority Scheduling

Preemptive · Lower number = Higher priority · Tie-breaking by Arrival Time then PID

PROCESS INPUT

Scenario A

Scenario B

Scenario C

Scenario D

Clear

PID	ARRIVAL	BURST	PRIORITY
P1	0	4	0
P1	2	3	2
P3	-1	5	1
P4	3	6	1

+ Add Process

▶ Run Priority

Row 1 [P1]: Priority must be ≥ 1 (got 0)

Row 2 [P1]: Duplicate PID "P1" - PIDs must be unique

Row 3 [P3]: Arrival time cannot be negative (got -1) - must be ≥ 0

ALGORITHM RULES

Tie-Breaking Rule:
When two processes have the *same priority*, the one with the **earlier arrival time** runs first. If arrival is also equal, lower PID wins.

Preemption: A running process is preempted when a higher-priority process arrives.

OS Scheduler

Priority

SRTF

Comparison

Algorithm B

SRTF Scheduling

Preemptive · Shortest Remaining Time First · Immediate preemption on new arrival

PROCESS INPUT

Scenario A

Scenario B

Scenario C

Scenario D

Clear

PID	ARRIVAL	BURST
P1	0	4
P1	2	3
P3	-1	5
P4	3	6

+ Add Process

▶ Run SRTF

Row 2 [P1]: Duplicate PID "P1" - PIDs must be unique

Row 3 [P3]: Arrival time cannot be negative (got -1) - must be ≥ 0

ALGORITHM RULES

Tie-Breaking Rule:
When two processes have the *same remaining time*, the one with the **earlier arrival time** continues (or wins). If equal, lower PID wins.

Preemption: When a new process arrives with a shorter remaining time than the current running process, it immediately preempts it. Preemption points are marked on the Gantt chart.